

Extending Clang API Notes for C++:

Overload-Specific Annotations for Ordinary Methods

- **Applicant:** Dominic Stöcker
 - **GitHub:** github.com/StoeckOverflow
 - **Discourse:** discourse.llvm.org/u/stoeckoverflow
- **Organization:** LLVM / Clang
- **Mentors:** John Hui, Egor Zhdan, Gábor Horváth
- **Intended scope:** Small project

Abstract

API Notes allow annotations to be attached to C++ declarations without modifying their headers, but they currently cannot distinguish between overloaded methods such as `foo(int)` and `foo(double)`. As a result, a single annotation applies to all overloads of a method name, limiting their usefulness in real-world APIs. This project introduces overload-specific matching for ordinary C++ methods while preserving existing API Notes behavior. The committed implementation proceeds in two phases:

- **Phase 1:** distinguishes overloads by explicit parameter types
- **Phase 2:** extends this to cv/ref-cases such as `foo()` versus `foo() const` and `foo() &` versus `foo() &&`

Template functions and mixed overload sets are discussed as possible follow-up work.

The result is a more precise and expressive system that allows annotations to target individual overloads without breaking existing usage.

Motivation

API Notes are sidecar YAML files that Clang reads while building or importing modules. They attach extra annotations to declarations without modifying the original headers. This is especially useful for interoperability work, where upstream headers often cannot or should not be edited directly. The key limitation is that API Notes cannot distinguish overloads such as `setValue(int)` and `setValue(double)`.

C++ Declaration	API Notes (YAML)
<pre>struct Widget { void setValue(int value); void setValue(double value); };</pre>	<pre>Tags: - Name: Widget Methods: - Name: setValue Parameters: - Position: 0 Type: int SwiftName: set(intValue:) - Name: setValue Parameters: - Position: 0 Type: double SwiftName: set(doubleValue:)</pre>

This is a natural way to express overload-specific Swift names in API Notes, but it does not work today. Current API Notes matching uses only the enclosing context and method name, so the two `setValue` entries are not treated as distinct overload selectors. In practice, this prevents assigning different `SwiftName` annotations to `setValue(int)` and `setValue(double)` through API Notes.

In cases like this, annotations such as `SwiftName` (the imported Swift-facing name) or `availability` cannot reliably distinguish between overloads if matching uses only the surrounding context and method name. As a result, a note written for `setValue` effectively applies to every method with that name, even when different overloads require different annotations. In practice, different overloads often require different `SwiftName` mappings or `availability` annotations, so treating them as a single unit limits real-world interoperability.

Making that possible without breaking existing API Notes usage improves:

- Swift interoperability customization for C++ APIs
- Clang-based tooling that consumes API Notes annotations
- correctness and maintainability of sidecar annotations for third-party headers

Limitations of the Current Matching Model

Today, API Notes do not carry enough information to say which declaration a note applies to when several methods share the same name. Declarations that share a method name inside one record are treated as the same match before parameter information can help tell them apart. In API Notes, parameter entries may refer to source-level parameters or to the implicit object parameter (`this` at position `-1`). However, this parameter information still does not participate in matching a note to a declaration, so it cannot distinguish cases such as `setValue(int)` versus `setValue(double)`, `status()` versus `status() const`, or `consume() &` versus `consume() &&`.

For example:

C++ Declaration	API Notes (YAML)
<pre>struct Widget { void setValue(int value); void setValue(double value); int status(); int status() const; int consume() &; int consume() &&; };</pre>	<pre>Tags: - Name: Widget Methods: - Name: setValue Parameters: [{ Position: 0, Type: int }] - Name: setValue Parameters: [{ Position: 0, Type: double }] - Name: status Parameters: [{ Position: -1, Type: "Widget *" }] - Name: status Parameters: [{ Position: -1, Type: "const Widget *" }] - Name: consume Parameters: [{ Position: -1, Type: "Widget &" }] - Name: consume Parameters: [{ Position: -1, Type: "Widget &&" }]</pre>

The basic processing model for .apinotes files is:

YAML -> binary tables -> match notes to declarations -> AST annotations

With richer method information such as the surrounding context, method name, and parameter information, these notes become distinguishable when matching them to declarations.

Increasing Difficulty of Overload-Specific Matching

This proposal addresses overload-specific matching in order of increasing difficulty:

1. methods that differ by explicit parameter types, such as `setValue(int)` and `setValue(double)`
2. methods that differ only by the implicit object parameter, such as `status()` and `status() const`, or `consume() &` and `consume() &&`
3. template and mixed overload sets, where one method name may refer to a function template and a non-template overload

The implementation plan follows this capability ladder: Phase 1 covers the first class, Phase 2 extends to the second, and templates remain possible follow-up work beyond the committed project scope.

Roadmap

This roadmap is organized by capability level first and implementation phase second.

- **Level 1** is implemented in Phase 1 and solves the overload-collapse problem for ordinary member overloads without breaking existing notes
- **Level 2** is planned for Phase 2 and extends the same model to receiver-sensitive member overloads, including cv- and ref-qualifiers
- **Possible Follow-Up** covers template functions and mixed overload sets beyond the committed implementation scope

Throughout the proposal, I use one running example to show increasingly demanding forms of overload-specific matching:

```
struct Widget {  
    void setValue(int value);  
    void setValue(double value);  
    int status();  
    int status() const;  
    int consume() &;  
    int consume() &&;  
};
```

Level 1: Explicit-Parameter Overloads

Implemented in Phase 1. This level gives users the missing core capability of distinguishing non-template member overloads by explicit parameter types. It reuses the current YAML shape and extends the C++ method lookup path so that explicit parameter types can participate in overload-specific matching.

This proposal is designed to preserve the behavior of existing API Notes. Notes that do not provide enough information to identify a single overload remain name-based and continue to apply to all matching declarations, just as today. In particular, notes without a Parameters section, notes with incomplete or annotation-only parameter information, and existing annotations without an overload-specific signature all stay on the legacy path. Overload-specific matching should only be enabled when a note provides enough information to identify one declaration, so existing API Notes remain valid and do not silently change behavior.

A note should only be treated as overload-specific when its explicit parameter typing is sufficient to identify a unique overload, otherwise it remains a legacy note. In the Widget example, this means that the two setValue notes are treated as overload-specific, while a note without Parameters would still apply to both overloads. This preserves current behavior for unique methods, annotation-only Parameters entries, and Position: -1 annotations. Phase 1 intentionally matches by normalized spelling, because this solves the common overload case with minimal complexity and avoids introducing semantic ambiguity or performance overhead in the initial implementation. As a result, straightforward formatting differences such as int* versus int * are handled.

Deliverable: A working overload-aware C++ API Notes pipeline for explicit-parameter overloads, with regression coverage for supported behavior, compatibility, and diagnostics

Definition of Done:

- non-template member overloads can be distinguished by explicit parameter types
- overloaded operators that differ by explicit parameter types are covered
- legacy notes remain compatible when no overload-specific signature is present
- duplicate typed notes are diagnosed
- typed notes that name no matching overload are a desired Phase 1 diagnostic improvement
- regression tests cover supported behavior and documented Phase 1 boundary cases

Behavior on the Widget example:

C++ Declaration	API Notes (YAML)
<pre>struct Widget { void setValue(int value); void setValue(double value); };</pre>	<pre>Tags: - Name: Widget Methods: - Name: setValue Parameters: - Position: 0 Type: int SwiftName: set(intValue:) - Name: setValue Parameters: - Position: 0 Type: double SwiftName: set(doubleValue:)</pre>

After Phase 1, the two `setValue` overloads become distinguishable. The receiver-sensitive `status` and `consume` cases still do not, because `Position: -1` is not yet part of lookup.

Level 2: Receiver-Sensitive Overloads

Planned for Phase 2. This level extends the same model to non-template member functions that differ through the implicit object parameter. This includes both non-cv-qualified versus cv-qualified overloads, as in `status()` versus `status() const`, and ref-qualified overloads such as `&` versus `&&`. In this phase, receiver-side information such as `Position: -1` participates in selection instead of remaining annotation-only.

Deliverable: Support for distinguishing non-template member overloads such as `int status()` and `int status() const`, as well as `int consume() &` and `int consume() &&`, by incorporating the implicit object parameter into overload selection

Definition of Done:

- non-cv-qualified versus cv-qualified member overloads can be distinguished
- ref-qualified member overloads can be distinguished
- `Position: -1` participates in lookup for this case
- regression tests cover supported behavior and explicit Phase 2 boundaries

Behavior on the Widget example:

C++ Declaration	API Notes (YAML)
<pre>struct Widget { int status(); int status() const; int consume() &; int consume() &&; };</pre>	<pre>Tags: - Name: Widget Methods: - Name: status Parameters: - Position: -1 Type: Widget * SwiftName: refreshStatus - Name: status Parameters: - Position: -1 Type: "const Widget *" SwiftName: status - Name: consume Parameters: - Position: -1 Type: "Widget &" SwiftName: consumeLValue - Name: consume Parameters: - Position: -1 Type: "Widget &&" SwiftName: consumeRValue</pre>

After Phase 2, both the `status()` / `status() const` and the `consume() &` / `consume() &&` overloads become distinguishable through the implicit object parameter.

Possible Follow-Up: Template Functions and Mixed Overload Sets

This remains possible follow-up work beyond the committed project scope. The most natural extension beyond ordinary non-template member overloads is template functions. A future design could begin with simple mixed overload sets in which one method name refers both to a function template and to a non-template overload. A natural first step would be to support matching against the primary template only, without considering explicit specializations or explicit template arguments, and only later extend this to more general template cases. This remains outside the committed implementation scope, but it is the clearest follow-up direction once Phases 1 and 2 are in place.

For example:

```
struct Widget {
    template <typename T>
    void setValue(T value);

    void setValue(int value);
};
```

This is the easier template case, because one name refers to a function template and a non-template overload without yet introducing explicit specializations. It is therefore a more realistic stretch goal than the more general template-matching problem.

Milestones

- **midterm milestone:** explicit-parameter overload-aware lookup for C++ API Notes is implemented, tested, compatibility-preserving, and clearly scoped
- **final milestone:** support for receiver-sensitive member overload differentiation, including cv-qualified and ref-qualified cases, is implemented, tested, and the Phase 1 and 2 patch series is ready for or already in upstream review

Risks and Scope Control

- the committed scope of this small project is Phase 1 plus the Phase 2 extension for receiver-sensitive member overloads, including cv- and ref-qualifiers
- template functions and mixed overload sets are the main possible follow-up work beyond the committed implementation
- typedef/type-alias equivalence is intentionally excluded from the proposal scope
- if review latency forces prioritization, Phase 1 remains the minimum success criterion

Broader Overload Landscape and Scope

This proposal focuses on C++ member functions first, and the scope boundaries should be explicit. Beyond the committed non-template work, template cases also have their own progression of difficulty:

1. **Ordinary non-template member overloads** are the simplest and most common case. They can be distinguished by explicit parameter types and form the core target of this proposal.
2. **Receiver-sensitive overloads** use the same explicit parameter list but differ through the implicit object parameter. They fit the same overall model once receiver information participates in matching.
3. **Simple mixed template/non-template overloads** are the most plausible stretch case, where one method name refers to a function template and a non-template overload.
4. **More general template cases** are harder, especially when explicit specializations and explicit template arguments are involved.

The simpler `setValue(T) / setValue(int)` pattern is the most reasonable place to begin if template support is ever attempted. More general template matching becomes harder once explicit specializations and explicit template arguments enter the picture. For example:

```
struct Widget {
    template <typename T> T convert(T value) { return value; }
    template <> int convert(int value) { return -value; }
    float convert(int value) { return value; }
};

auto x = widget.convert(4.2);      // convert<double>() -> double
auto y = widget.convert(4);        // convert() -> float
auto z = widget.convert<int>(4);    // convert<int>() -> int
```

These cases are harder because a single apparent method name may refer to a primary template, one or more explicit specializations, and non-template overloads at the same time. Supporting them therefore raises additional declaration-identity questions beyond ordinary methods, such as whether template arguments, specialization state, or more canonicalized signature information must also participate in matching.

Accordingly, this proposal deliberately focuses on non-template member overloads first. Phase 1 addresses ordinary overloads, and Phase 2 extends that model to receiver-sensitive cases, leaving template support as future work.

As a result, this project should be understood as enabling precise notes for ordinary member overloads first, not as solving all forms of C++ overload resolution in API Notes. This keeps the initial design simple and reviewable, while leaving a clear path for a principled extension to template cases in future work.

For type spellings, the project scope is:

- **Formatting differences:** in Phase 1, for example `int *` versus `int*`

```
- Name: setBuffer
  Parameters: [{ Position: 0, Type: "int *" }]
```

should still match `void Widget::setBuffer(int*);`.

- **Exact complex spellings:** in scope as long as the YAML and declaration use the same spelling, for example `Outer::Inner`, `Foo<Bar, 42>`, and `int[32]`

```
- Name: configure
  Parameters: [{ Position: 0, Type: "Outer::Inner" }]
```

is intended to work when the declaration uses the same spelling.

- **Typedef/type-alias equivalence:** explicitly out of scope for this proposal

C++ Declaration	API Notes (YAML)
<pre>using Count = int;</pre>	<pre>- Name: resize Parameters: [{ Position: 0, Type: int }]</pre>
<pre>struct Widget { void resize(Count value); };</pre>	

This kind of equivalence is intentionally excluded from the proposal, because alias-based matching can make overload selectors harder to read and reason about, while also adding non-trivial implementation complexity. Unless there is a strong concrete use case, the proposal should not commit to this behavior.

Diagnostics and Failure Modes

Giving users more precise overload selection also creates more room for error and fragility, so the user model should fail in understandable ways. When an API note is specific enough to name one overload, it should either match that overload or make it clear why it did not.

Phase 1 commits to three concrete behaviors: duplicate typed notes are diagnosed, incomplete or annotation-only parameter notes stay on the legacy path, and typed notes that clearly name one overload but match none produce a targeted diagnostic.

Duplicate notes should still be diagnosed:

```
- Name: setValue
  Parameters: [{ Position: 0, Type: int }]
- Name: setValue
  Parameters: [{ Position: 0, Type: int }]
```

Incomplete notes should stay on the legacy path rather than pretending to be precise selectors:

```
- Name: setValue
  Parameters:
  - Position: 0
    Nullability: N
```

Refactors can also invalidate typed notes:

```
// header after refactor
struct Widget {
    void setValue(long value);
};

# older API notes
- Name: setValue
  Parameters: [{ Position: 0, Type: int }]
```

Typos or unsupported typed notes can fail to match even when the user intended to select a specific overload:

```
- Name: setValue
  Parameters: [{ Position: 0, Type: double }]
```

Likewise, a note may look plausible but still fail to match because it names a different type than the header actually uses:

C++ Declaration	API Notes (YAML)
<pre>struct Widget { void setValue(double value); };</pre>	<pre>- Name: setValue Parameters: [{ Position: 0, Type: long }]</pre>

Phase 1 will preserve existing duplicate diagnostics and compatibility-first fallback behavior for legacy notes. When a typed note implies overload selection, it should not fail silently. At minimum, the diagnostic should explain that no overload with the requested parameter types was found for the named method, for example:

```
error: no overload of 'setValue' matches parameter types '(int)'
note: candidate overloads are '(double)' and '(long)'
```

Where feasible, it should also point to nearby candidate overloads so users can recover from typos, stale notes after refactoring, or unsupported type spellings more easily.

Current Prototype and Implementation Status

I have already developed a local prototype for the Phase 1 direction. The current prototype is available on my fork in the branch `clang/apinotes-cxx-overloads`. It demonstrates explicit-parameter overload-aware matching, preserves the YAML surface syntax, validates typed-vs-legacy coexistence, and makes the Phase 2 extension plausible. It is intentionally incomplete and mainly serves to validate feasibility and guide upstreamable design decisions.

The prototype changes the declaration identity used for C++ methods:

old C++ method identity:
(parent context, method name)

Phase 1 prototype identity:
(parent context, method name, key kind, explicit parameter-type sequence)

key kind:
typed/signature-specific
legacy/name-only

The prototype establishes the following Phase 1 behavior:

- typed/signature-specific C++ method notes are distinguished from legacy/name-only notes
- lookup is typed first, then legacy fallback
- explicit Parameters: [] remains distinct from omitted Parameters
- Position: -1 is preserved as receiver-side annotation but does not yet participate in overload identity
- duplicate effective C++ method notes in one tag context are diagnosed

It also has the following intentional Phase 1 limitations:

- matching is based on normalized type spellings rather than semantic type equivalence
- Position: -1 does not yet participate in method identity

- conversion operators are future work rather than part of the committed prototype scope
- some tests are intentionally boundary or limitation tests rather than success-case coverage

At lookup time, SemaAPINotes computes the same normalized explicit-parameter signature from CXXMethodDecl, and the reader preserves compatibility with older name-only behavior:

compute normalized explicit parameter types from CXXMethodDecl

lookup C++ method note:

1. try exact typed/signature-specific match
2. if no typed match exists, try legacy name-only match

These tests define the Phase 1 compatibility contract and its boundaries.

Overload-specific matching

C++ Declaration	API Notes (YAML)
<pre>struct S { void foo(int); void foo(double); };</pre>	<pre>Tags: - Name: S Methods: - Name: foo Parameters: [{ Position: 0, Type: int }] SwiftName: fooInt - Name: foo Parameters: [{ Position: 0, Type: double }] SwiftName: fooDouble</pre>

This demonstrates the intended Phase 1 behavior: methods that share the same name within one record are distinguished by their explicit parameter-type sequences, enabling overload-specific annotations. This behavior is covered by tests such as overload-param-types.cpp and overload-operator-plus.cpp.

Typed vs legacy compatibility

- ```
- Name: foo
 Availability: unavailable

- Name: foo
 Parameters: [{ Position: 0, Type: int }]
 SwiftName: fooInt

- Name: ping
 Availability: unavailable

- Name: ping
 Parameters: []
 SwiftName: pingExact
```

This behavior is covered by tests such as typed-vs-legacy.cpp, yaml-convert-diags-cxx-methods.cpp, and yaml-roundtrip-3.test, which exercise coexistence of typed and legacy entries and ensure that omitted Parameters and explicit Parameters: [] remain distinguishable.

### Intentional boundaries

```
struct S {
 void foo();
 void foo() const;
};
```

These limitations are covered by tests such as overload-conversion.cpp and overload-const-qual.cpp, which document that conversion operators and cv-qualified overload identity are intentionally out of scope for this phase.

## Failure-mode coverage

```
- Name: frob
 Parameters: [{ Position: 0, Type: int }]
- Name: frob
 Parameters: [{ Position: 0, Type: int }]
```

The prototype already diagnoses duplicate typed notes during YAML conversion, and the regression tests keep that behavior stable.

Another important compatibility case is that notes with incomplete or annotation-only parameter information stay in the legacy path rather than silently failing to match. `typed-vs-legacy.cpp` makes that behavior explicit.

## Timeline, Upstreaming Plan, and Deliverables

The project follows an 8-week coding period with review-driven upstreaming. The goal is to bring the Phase 1 and Phase 2 patch series into upstream-ready shape, reserving time for review iteration, cleanup, and documentation rather than additional committed feature work.

### Weeks 1–4: Phase 1 — Explicit-Parameter C++ Method Identity

Planned work:

- implement `CXXMethodTableKey` and integrate it across writer, reader, and `SemaAPINotes`
- add regression tests for overload matching and compatibility behavior
- refine patch structure based on upstream review

Planned patch flow:

- Patch 1: regression tests for overload matching and current limitations
- Patch 2: `CXXMethodTableKey` infrastructure and serialization changes
- Patch 3: `SemaAPINotes` integration and typed lookup with legacy fallback
- Patch 4: diagnostics, YAML classification clarifications, and cleanup

**Deliverable:** Overload-aware C++ API Notes lookup for explicit parameter types, with compatibility-preserving fallback and focused regression coverage

### Weeks 5–8: Phase 2 — Receiver-Sensitive Member Overloads

Planned work:

- extend method identity to support non-template member overloads that differ through the implicit object parameter, including cv- and ref-qualifiers
- implement receiver-side signature computation for cv-qualified and ref-qualified member overloads in `SemaAPINotes`
- add support for receiver-sensitive overload differentiation for those methods
- add targeted regression tests for implicit-object-sensitive lookup
- use remaining time for review iteration and cleanup

Possible patch flow:

- Patch 5: extend method key to model implicit object parameter identity
- Patch 6: sema-side computation and lookup integration for receiver-sensitive methods
- Patch 7: additional tests, cleanup, and review-driven refinement

**Deliverable:** Support for receiver-sensitive member overload differentiation in C++ API Notes by incorporating the implicit object parameter into method identity

### Scope Control

- the committed scope is Phase 1 plus the Phase 2 extension for receiver-sensitive member overloads, including cv- and ref-qualifiers
- template functions and mixed overload sets are intentionally left as future work beyond the committed small-project schedule
- typedef/type-alias equivalence remains explicitly outside the committed implementation scope

## Benefits to the LLVM Community

This project lets API Notes target individual C++ overloads instead of forcing one note to apply to every method with the same name. In practice, this allows annotations such as `SwiftName` or `availability` to be attached to the intended overload, rather than being forced to apply uniformly across all overloads.

It also establishes a path toward handling receiver-sensitive overloads and, later on, template-related cases without breaking existing notes. That makes API Notes more precise for real libraries while keeping the initial project scope small and reviewable.

## About Me

### Background and Relevant Experience

I am currently pursuing an M.Sc. in Computer Science at TU Berlin (German grade average 1.2) and work as a student research assistant at the Chair for Compiler and Programming Languages. My research focuses on integrating value-dependent types into MLIR as part of my Master's thesis, which involves extending type systems, designing IR-level invariants, and reasoning about canonical type identity.

Previously, I contributed to the [DAPHNE](#) project, where I implemented MLIR passes and developed a prototype for dynamic recompilation in C++. That work strengthened my ability to navigate large compiler codebases, implement transformations, and follow incremental development and testing practices. I also supported a Compiling Techniques course by maintaining and extending a teaching compiler, further deepening my understanding of semantic analysis and compiler pipelines.

To prepare for this project, I built Clang locally, ran the API Notes regression suite, studied relevant components such as `APINotesReader`, `APINotesWriter`, `APINotesYAMLCompiler`, and `SemaAPINotes`, and implemented a prototype for overload-aware C++ API Notes lookup.

My primary programming languages are C++, Scala, and Python, with additional experience in Rust. My C++ experience mainly comes from compiler-related systems and MLIR-based infrastructure.

### Motivation and Project Fit

My academic work centers on compiler infrastructure, type systems, and intermediate representations, making LLVM a natural fit for my current work and long-term goals. I plan to pursue a career in compiler engineering and a PhD in this area, and contributing to LLVM is an opportunity to work on production-grade compiler components within a highly active ecosystem.

This project is particularly well aligned with my background because it focuses on declaration identity, semantic matching, and the incremental evolution of compiler infrastructure.

I am also strongly interested in language interoperability, especially between C++ and Swift. API Notes play a key role in integrating existing C++ libraries into Swift without modifying upstream headers. Extending them to support precise overload-specific annotations improves both correctness and usability in real-world APIs.

Overall, this project combines practical engineering work with questions about type systems and compiler design, making it an excellent match for my interests and experience.

### Availability and Commitments

During the GSoC period, I will be working on my Master's thesis, which has a fixed submission date of July 9, 2026. The thesis is closely related to LLVM and compiler infrastructure, creating strong overlap with this project.

I expect to divide my time during the early coding period, but after submission I will be able to fully focus on the GSoC project. The proposal is scoped such that the core Phase 1 deliverable, and potentially the Phase 2 extension, are achievable within this timeline.

I will communicate regularly with mentors and adapt the schedule as needed to ensure steady progress and successful completion of the project.